Synthetic Data Generation for Healthcare: Generative Model Analysis

By: Javi Wu, Matthew Huertas, Mishel Ghukasyan, Stella Sinlao, Stephen Patpatyan

The Problem

- Regulations like HIPAA severely restrict sharing real patient data, blocking ML research and model development.
- Researchers need large, diverse datasets - especially for rare conditions and underrepresented patient subgroups.

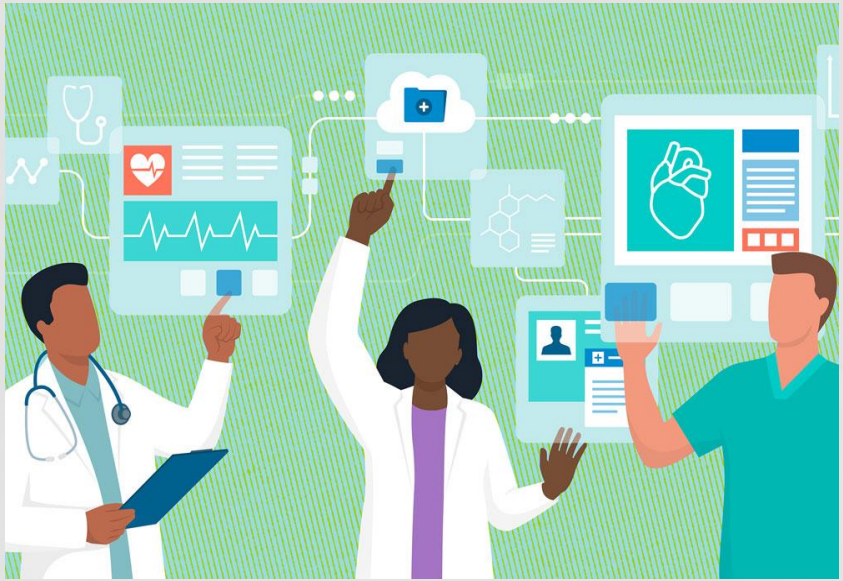


Image Source: https://news.mit.edu/2022/patient-data-risks-low-1006

The Solution

- **Our project aims to compare the various generative models with their accuracy in creating synthetic healthcare data**

- **Columns used: Age, Gender, LOS, ICU, Primary and secondary diagnoses, multiple lab events**

Experimental Procedure

- Cleaned MIMIC-III Data (https://physionet.org/content/mimiciii/1.4/)
- Main output across all models are LABEVENTS values
- LOS bucketed into categories: <2d, 2-7d, 7-14d, >14d
- Primary ICD-9 mapped to 19 CCS disease categories
- 31 binary Elixhauser comorbidity flags from all secondary diagnoses.
- Only labs in window 6hr before to 24hr after first ICU admission.
- 7 right-skewed labs log-transformed
- StandardScaler normalization after transforms, BatchNorm after training
- Clip physiologically implausible values and gradient clipping

Conditional Variational Autoencoder (cVAE)

Variant 1: Standard (KL)

- Encoder outputs mu and logvar per patient
- Sampling: mu + eps * exp(0.5 * logvar) -> Reparameterization Trick
- Loss: 0.5 * MSE + 0.5 * MAE + beta * KL
- Beta max: 0.5 | Cycle: 75 epochs
- KL = -0.5 * mean(1 + logvar - mu^2 - exp(logvar))

Variant 2: Sliced Wasserstein Distance

- Encoder outputs z directly
- Sampling: none since z is deterministic
- Loss: 0.5 * MSE + 0.5 * MAE + beta * SWD
- Beta Max: 1.0 | Cycle: 75 epochs | Projections: 50
- SWD = sorted 1D Wasserstein averaged over 50 random directions

Both go through the decoder with 2 fully connected layers with ReLU and batch normalization.

cVAE Pros and Cons

Variant 1: Standard (KL)

- Each patient gets its own distribution
- Fast to compute, no approximation
- Reparameterization trick is elegant and stable

- KL can become infinite
- Forces every patient toward N(0,1)
- Mean-regression bias

Variant 2: Sliced Wasserstein Distance

- SWD always finite
- Aggregate regularization
- Better tail coverage

- Approximate distance with n_projections
- Less support for small batch sizes
- Deterministic encoder, where stochasticity is only from prior SWD

OVERALL: cVAEs uniquely combine controllable generation with probabilistic inference, letting you both generate new samples conditioned on specific attributes and encode real data into a structured latent space, but is also prone to posterior collapse where the model learns to ignore the latent space.

Autoregressive Model (ARM)

- Treats the 23 labs as an ordered sequence, explicitly learning how each lab depends on both the patient's background and all previously predicted labs.
- **Sequential Rollout**: Predicts labs one by one in a fixed clinical sequence (e.g., Glucose -> Sodium -> Potassium).
- **Contextual Inputs**: At each step i, the model receives the 60-dim patient condition vector, the i already-predicted labs, and a binary mask.
- **Weight Sharing**: A single ARMStepNet is shared across all 23 steps to reduce parameters and force generalization across the sequence.

Generator Architecture

- **Input Construction**: Condition vector + padded past labs + progress mask
- **Hidden Structure**: Shared 3-layer MLP (256→256→128) with BatchNorm, ReLU, and 0.1 dropout
- **Output Layer**: Outputs one scalar lab prediction for the current step

Training Strategy

- **Teacher Forcing**: Uses real past lab values to stabilize learning
- **Loss Function**: Minimizes a balanced mix of MSE and MAE to handle both accuracy and outliers

ARM Pros and Cons

- Pros
  - Explicitly models clinical correlations between labs
  - Weights are shared across steps to force generalization
  - Strictly respects physiological bounds
- Cons:
  - Slower inference due to sequential rollout (23 passes per patient)
  - Prone to Exposure Bias where errors compound as the sequence progresses

Probabilistic Conditional ARM (PCARM)

- Builds on the ARM concept but predicts a distribution, mean and variance, for each lab
- At generation time, it will sample from the predicted distribution to produce diverse synthetic records
- Designed in order to address variance collapse / mean regression seen in deterministic ARM results

Generator Architecture

- **Input**: Previous lab value + encoded patient condition vector at each step
- **Hidden Structure**: 2 layer LSTM (256 hidden units) with condition initialized hidden state
- **Output Layer**: Outputs mu and log-variance per step, defining a Gaussian distribution

Training Strategy

- **Teacher Forcing**: Uses real past lab values to stabilize learning
- **Loss Function**: Gaussian negative log-likelihood which will penalize both inaccuracy and overconfidence
- **Generation**: Samples from predicted distribution instead of outputting a single point estimate

PCARM Pros and Cons

- Pros
  - Reduces variance collapse by sampling from learned distributions instead of just predicting the mean
  - LSTM hidden state carries dependencies between labs without explicit mask
  - Can compare stochastic, sampled, vs deterministic, mu only, output to measure the effect of probabilistic modeling
- Cons:
  - Larger model, ~1M params vs ~127k params, which is slower to train
  - Still prone to exposure bias where errors compound across the sequence
  - Sampling may weaken correlation preservation between labs compared to point estimates

Generative Adversarial Network (GAN)

- GAN is commonly used for synthetic data generation.
- Due to range of feature values, GAN needed PowerTransformer so the discriminator didn’t just get stuck at 0.
- Zero-sum “game” between Generator (G) and Discriminator (D) -> G wants to minimize the chance of D correctly identifying fake data while D wants to maximize the probability of assigning the correct label to both real and fake data

Generator Class

- Input: latent vector (noise) sampled from prior distribution
- Structure: Series of fully connected layers, LeakyReLU to prevent gradient vanishing
- Output Layer: Uses Sigmoid activation function to match normalized range of real lab_feature values

Discriminator Class

- Input: vector representing lab features (either real or synthetic)
- Structure: Dense layers with LeakyReLU activations. Dropout layers to prevent the discriminator from becoming too strong too quickly.
- Output Layer: A single node with Sigmoid activation outputting a probability score (0 to 1)

GAN Pros and Cons

- Pros:
  - GAN, in general, is effective at modeling distributions of different feature values, making diverse synthetic data more realistic
  - Unlike some VAEs that assure data follows a specific distribution, GANs can learn the true shape of the data
- Cons:
  - The preprocessing to make a perfectly performing GAN model can be quite complex depending on what the structure of the data is
  - Mode collapse was a big issue with this model which is when the generator loses sense of the diversity in the dataset
    - To try to mitigate this issue, I had to use PowerTransformer rather than just log transformation due to high sensitivity to outliers

Recurrent Neural Network (RNN)

Masks 20% of each patient's 23 lab values and learns to reconstruct them using the remaining labs and patient context. Loss is only computed on masked positions.

Architecture

- **Input:** Masked lab value + mask flag + 60-dim condition vector
- **LSTM Layers:** 2 layers, hidden size 128, dropout 0.4
- **Output Layer:** Linear, one predicted value per step

Training Strategy

- **Loss Function:** MSE on masked positions only
- **Optimizer:** Adam | lr = 0.001 | weight decay = 1e-4
- **Early Stopping:** Patience 7 epochs, restores best weights

RNN Pros and Cons

Pros

- Learns cross-lab correlations without assuming temporal ordering
- Robust to missing lab values
- Reconstructs all 23 labs in a single forward pass
- Early stopping + dropout controlled overfitting (train/val gap: 0.50 -> 0.09)

Cons

- Does not model how lab values change over time
- High-variance labs like po2 and platelets remain hard to predict
- Lab sequence order is arbitrary

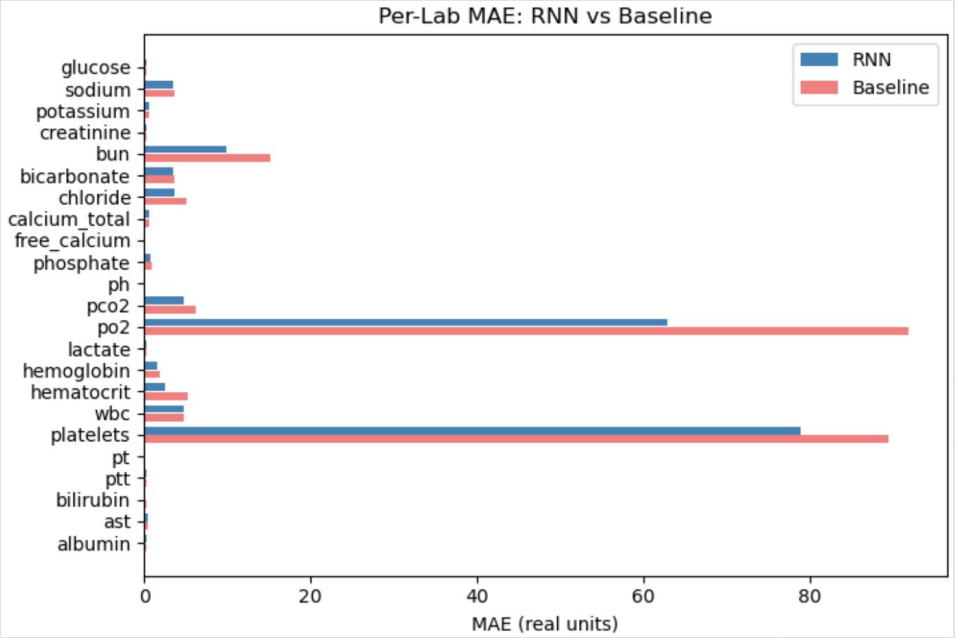
RNN Results

Key Metrics

Overall MAE: 7.73 real units (baseline: 10.10)
Improvement: 23% over mean baseline
Labs beaten: 20 of 23
Hematocrit MAE: 2.597 (range: 5-70)

Model Improvements

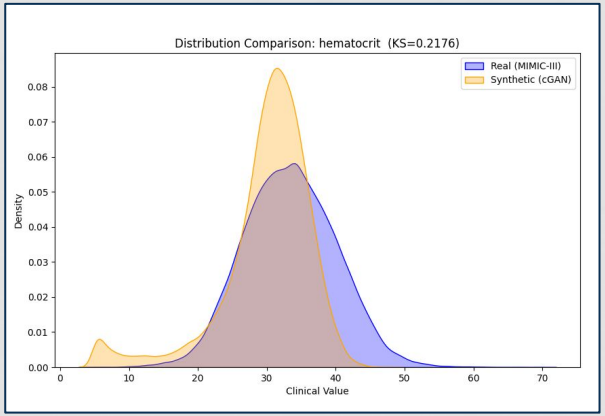
Initial model overfit (train/val gap 0.50). Dropout 0.4, weight decay, and early stopping reduced this to 0.09 and improved MAE from 8.37 to 7.73.



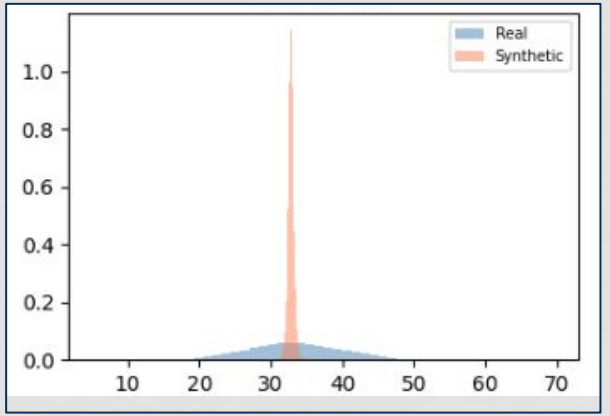
Model Accuracy Comparison

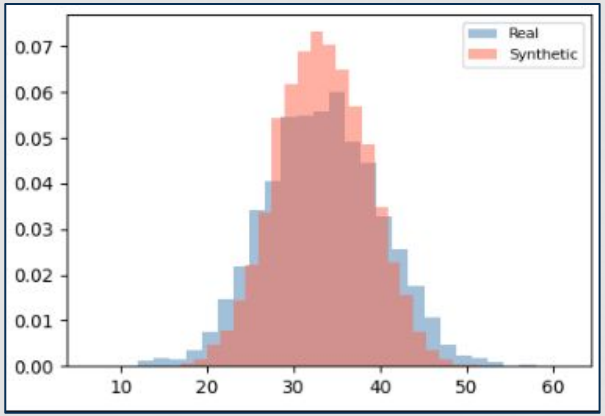
Comparison of Synthetic Data and Real Data for “hematocrit”

GAN

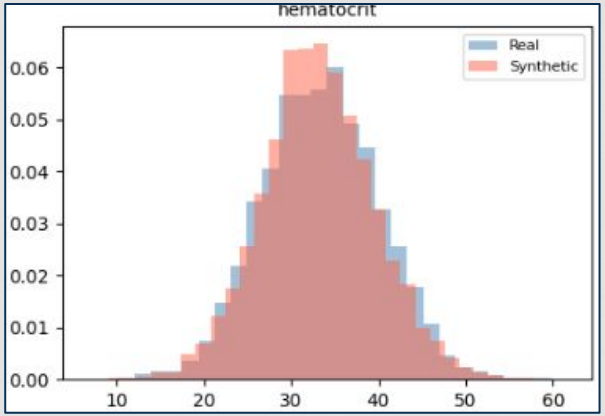


ARM

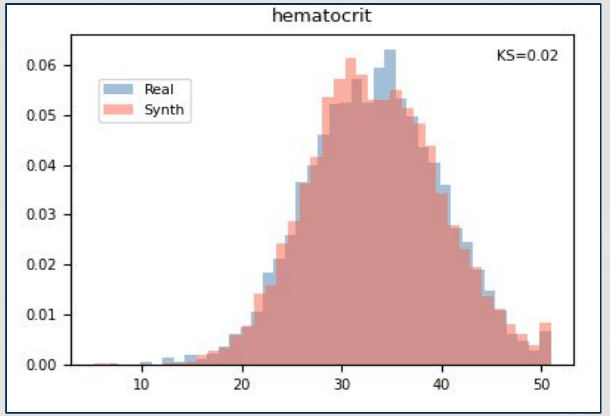




Standard cVAE



SWD cVAE



PCARM

KS Statistic across Models

KS statistic table

Feature	ARM LSTM	cVAE	SWD-cVAE	GAN	PCARM
glucose	1.0000	0.184	0.063	0.1499	0.048
sodium	0.4923	0.105	0.090	0.2163	0.078
potassium	0.4572	0.178	0.069	0.1275	0.089
creatinine	0.9708	0.165	0.145	0.1828	0.111
bun	0.5522	0.129	0.134	0.0978	0.070
bicarbonate	0.4901	0.153	0.076	0.3700	0.111
chloride	0.4749	0.099	0.049	0.3778	0.068
calcium_total	0.5486	0.113	0.095	0.2236	0.107
free_calcium	0.7528	0.299	0.326	0.5126	0.304
phosphate	0.4606	0.155	0.108	0.1708	0.133
ph	0.6845	0.218	0.185	0.2485	0.204
pco2	0.6023	0.231	0.244	0.3686	0.203
po2	0.6884	0.241	0.185	0.2056	0.285
lactate	0.9973	0.221	0.268	0.3739	0.278
hemoglobin	0.4589	0.068	0.042	0.1709	0.018
hematocrit	0.4499	0.063	0.041	0.2176	0.018
wbc	0.4715	0.150	0.074	0.1282	0.137
platelets	0.4688	0.122	0.069	0.2232	0.069
pt	1.0000	0.141	0.098	0.1100	0.138
ptt	1.0000	0.117	0.093	0.0621	0.115
bilirubin	0.9670	0.296	0.371	0.2949	0.347
ast	1.0000	0.284	0.289	0.4504	0.347
albumin	0.6767	0.338	0.335	0.3675	0.349

Conclusion - Models

- KL vs SWD for cVAE: Aggregate regularization gives encoder freedom to place extreme patients away from center which directly explains improved tail coverage
- GAN mode dropping: generator finds convincing regions and ignores others, producing PTT=0.062 (best in table) alongside bicarbonate=0.370 (near worst)
- PCARM captures correlated lab pairs more faithfully than latent space models by conditioning each lab on previously generated values
- ARM designed for sequential time-series prediction - it learns to predict the next value in a sequence given previous values, not to model a joint static distribution of 23 labs simultaneously

Conclusion - Data

- **free_calcium, lactate, bilirubin, AST, albumin, po2 score poorly across ALL five models**
    - This is definitive evidence of a data limitation not a model limitation
- **Data cleaning decisions such as removing double-log transform, adding log-transform, and removing column FLAG filtering had a far greater impact than any architecture decision**